UNIVERSIDAD DEL BÍO-BÍO

13 Programa final – Procesamiento de imágenes usando webcam

Tutorial Procesamiento de Imagen con webcam

Estudiantes Practicantes:
Javiera Gutiérrez
Javiera Henríquez
Teresa Vidal

Profesor:
Luis Vera

Laboratorio CIMUBB

2023-1

Procesamiento de imágenes usando webcam

Como proyecto final, se unirán los proyectos anteriores, creando un solo programa que contenga todo el análisis de imágenes.

Para ellos, creamos otro archivo .py y le añadimos las siguientes librerías.

Creamos nuestra ventana, y le añadimos tamaño, que no se modificable y un título.

Además, creamos las variables globales **Captura** y **CapturaG** que nos permitirán trabajar con las fotos a color y en escala de grises en distintas funciones a lo largo del código.

13 Programa final.py

13 Programa final.py > ...

```python
import tkinter as tk
from tkinter import *
from tkinter import ttk
from PIL import Image
from PIL import ImageTk
import imutils
import cv2

# Crea ventana, define tamaño y título
ventana = tk.Tk()
ventana.geometry("1320x800")
ventana.resizable(0,0)
ventana.title("Proyecto procesamiento de imagen con Webcam")

#Variables globales
global Captura, CapturaG
```

Añadimos nuestro código para iniciar la webcam.

```
#Inicia cámara web
def camara():
    global capture
    capture = cv2.VideoCapture(0)
    iniciar()

def iniciar():
    global capture
    if capture is not None:
        BCapturar.place(x=250,y=330,width=91,height=23)
        ret, frame = capture.read()
        if ret == True:
            frame = imutils.resize(frame, width=311)
            frame = imutils.resize(frame, height=241)
            ImagenCamara = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            im = Image.fromarray(ImagenCamara)
            img = ImageTk.PhotoImage(image= im)
            LImagen.configure(image= img)
            LImagen.image = img
            LImagen.after(10,iniciar)
        else:
            LImagen.image = ""
            capture.release()
```

También añadimos el código para tomar fotos usando la webcam, que vimos anteriormente en este tutorial.

```
def Capturar():
    global valor, Captura, CapturaG
    camara = capture
    return_value, image = camara.read()
    frame = imutils.resize(image, width=301)
    frame = imutils.resize(frame, height=221)
    CapturaG = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    Captura = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    im = Image.fromarray(Captura)
    img = ImageTk.PhotoImage(image= im)
    imG = Image.fromarray(CapturaG)
    imgG = ImageTk.PhotoImage(image= imG)
    GImagenROI.configure(image= imgG)
    GImagenROI.image = imgG
    LImagenRecorte.configure(image= img)
    LImagenRecorte.image = img
```

Añadimos el código para umbralizar una imagen a color usando los valores RGB y su análisis de manchas correspondiente.

```python
def rgb():
    global img_mask, img_aux, bin_imagen
    Minimos = (int(SRedI.get()),int(SGreenI.get()),int(SBlueI.get()))
    maximos = (int(SRedD.get()),int(SGreenD.get()),int(SBlueD.get()))
    img_mask = cv2.inRange(ImgRec, Minimos, maximos)
    img_aux = img_mask
    img_mask = Image.fromarray(img_mask)
    img_mask = ImageTk.PhotoImage(image= img_mask)
    LImagenManchas.configure(image=img_mask)
    LImagenManchas.image = img_mask
    _, bin_imagen = cv2.threshold(img_aux, 0, 255, cv2.THRESH_BINARY_INV)

def manchas():
    # Contar el número de píxeles con manchas
    num_pixels_con_manchas = cv2.countNonZero(bin_imagen)
    # Calcular el porcentaje de manchas
    porcentaje_manchas = 100 - (num_pixels_con_manchas / bin_imagen.size) * 100
    #Contornos
    contornos = cv2.findContours(img_aux, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[0]
    #Cantidad de contornos
    num_formas = len(contornos)
    Cadena = f"Cantidad de manchas blancas: {num_formas}\nPorcentaje área con manchas: {round(porcentaje_manchas,2)}%"
    CajaTexto2.configure(state='normal')
    CajaTexto2.delete(1.0, tk.END)
    CajaTexto2.insert(1.0, Cadena)
    CajaTexto2.configure(state='disabled')
```

Hacemos lo mismo para la Umbralización de una imagen en escala de grises, y para su análisis de manchas.

```python
def umbralizacion():
    global thresh1, mask
    valor = int(numeroUmbra.get())
    ret, thresh1 = cv2.threshold(CapturaG, valor, 255, cv2.THRESH_BINARY)
    Umbral = Image.fromarray(thresh1)
    Umbral = ImageTk.PhotoImage(image=Umbral)
    UImagen.configure(image = Umbral)
    UImagen.image = Umbral

    min = (valor, valor, valor)
    max = (255,255,255)
    mask = cv2.inRange(Captura, min, max)

def manchasG():
    # Contar el número de píxeles con manchas
    num_pixels_con_manchas = cv2.countNonZero(thresh1)
    # Calcular el porcentaje de manchas
    porcentaje_manchas = 100 - (num_pixels_con_manchas / thresh1.size) * 100
    #Contornos
    contornos = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[0]

    #Cantidad de contornos
    manchas = len(contornos)
    Cadena = f"Cantidad de manchas blancas: {manchas}\nPorcentaje área sin manchas:{round(porcentaje_manchas,2)}%"
    CajaTexto.configure(state='normal')
    CajaTexto.delete(1.0, tk.END)
    CajaTexto.insert(1.0, Cadena)
    CajaTexto.configure(state='disabled')
```

Además, añadimos el código para poder usar el evento del mouse y saber cuándo se hace clic derecho sobre la imagen, y de esa forma determinar sus coordenadas x e y.

Se añade el código para recortar una imagen de acuerdo a sus coordenadas x e y dadas por el usuario.

```
def mostrar_coordenadas(event):
    coordenadas['text']=f'x = {event.x}    y = {event.y}'

def recortar():
    global ImgRec
    Vx1 = int(x1.get())
    Vy1 = int(y1.get())
    Vx2 = int(x2.get())
    Vy2 = int(y2.get())
    ImgRec = Captura[Vx1:Vx2, Vy1:Vy2]
    Im = Image.fromarray(ImgRec)
    ImRec = ImageTk.PhotoImage(image=Im)
    LImagenROI.configure(image= ImRec)
    LImagenROI.image = ImRec

```

Añadimos el código para siete botones **camara, Capturar, rgb, manchas, umbralizacion, manchasG** y **recortar**¸ para hacer uso de las funciones que creamos.

```
#Botones
BCamara = tk.Button(ventana, text="Iniciar cámara", command=camara)
BCamara.place(x=60,y=330,width=90,height=23)
BCapturar = tk.Button(ventana, text="Tomar foto", command=Capturar)
BCapturar.place(x=250,y=330,width=91,height=23)
BManchas = tk.Button(ventana, text="Umbralización", command=rgb)
BManchas.place(x=760,y=640,width=100,height=23)
ManchasRGB = tk.Button(ventana, text="Análisis de manchas", command=manchas)
ManchasRGB.place(x=880,y=640,width=120,height=23)
BBinary = tk.Button(ventana, text="Umbralización", command=umbralizacion)
BBinary.place(x=800,y=310,width=90,height=23)
BManchasG = tk.Button(ventana, text="Análisis de Manchas", command=manchasG)
BManchasG.place(x=1100,y=310,width=131,height=23)
BRecortar = tk.Button(ventana, text="Recortar", command=recortar)
BRecortar.place(x=155,y=700,width=80,height=23)

```

Añadimos los **SpinBox** para poder seleccionar el número para la Umbralización en escala de grises, y para que el usuario seleccione las coordenadas x e y para recortar la imagen.

```
#SpinBox
numeroUmbra = tk.Spinbox(ventana, from_=0,to=255)
numeroUmbra.place(x=900, y=310, width=42, height=22)
x1 = tk.Spinbox(ventana, from_=0,to=298)
x1.place(x=140, y=630, width=42, height=22)
y1 = tk.Spinbox(ventana, from_=0,to=239)
y1.place(x=240, y=630, width=42, height=22)
x2 = tk.Spinbox(ventana, from_=1,to=298)
x2.place(x=140, y=660, width=42, height=22)
y2 = tk.Spinbox(ventana, from_=1,to=239)
y2.place(x=240, y=660, width=42, height=22)

#Label
LRed = tk.Label(ventana, text="R")
LRed.place(x=530,y=640,width=21,height=16)
LGreen = tk.Label(ventana, text="G")
LGreen.place(x=530,y=680,width=21,height=16)
LBlue = tk.Label(ventana, text="B")
LBlue.place(x=530,y=720,width=21,height=16)
coordenadasTitulo = tk.Label(ventana, text="Coordenadas")
coordenadasTitulo.place(x=505, y=310)
coordenadas = tk.Label(ventana, text="")
coordenadas.place(x=495, y=330)
Lx1 = tk.Label(ventana, text="x1")
Lx1.place(x=120, y=630)
Ly1 = tk.Label(ventana, text="y1")
Ly1.place(x=220, y=630)
Lx2 = tk.Label(ventana, text="x2")
Lx2.place(x=120, y=660)
Ly2 = tk.Label(ventana, text="y2")
Ly2.place(x=220, y=660)
```

Se añade un código para poder mostrar el logo de la Universidad dentro de la ventana. La imagen solo se mostrará si se encuentra en la misma carpeta donde está contenido el código.

```
#Logo Universidad
logo = tk.PhotoImage(file="LogoUBB.png")
logoUBB = ttk.Label(image=logo)
logoUBB.place(x=1250, y=615)
```

Añadimos otros textos por pantalla, pedidos por el profesor, los que son: nuestros nombres, nuestra carrera, el nombre del profesor y el nombre del Laboratorio CIM.

```
#Nombres alumnas - carrera - profesor - Lab CIM
alumnas = tk.Label(ventana, text="Estudiantes practicantes\n\nJaviera Gutiérrez\nJaviera Henríquez\nTeresa Vidal").place(x=1075, y=620)
carrera = tk.Label(ventana, text="Ingeniería de Ejecución\nen Computación e Informática").place(x=1060, y=720)
profesor = tk.Label(ventana, text="Profesor\nLuis Vera").place(x=1250, y=700)
LabCIM = tk.Label(ventana, text="Lab CIM").place(x=1250, y=740)
```

Añadimos seis cuadros grises, donde se van a contener nuestras imágenes y los trabajos que haremos sobre ellas.

Se añaden dos cuadros de texto, donde se verá el resultado del análisis de manchas tanto de la imagen en escala de grises, como la RGB.

```
#Cuadros de Imagen grises
LImagen = tk.Label(ventana, background="gray")
LImagen.place(x=50,y=50,width=300,height=240)
LImagenROI = tk.Label(ventana, background="gray")
LImagenROI.place(x=390,y=380,width=300,height=240)
GImagenROI = tk.Label(ventana, background="gray")
GImagenROI.place(x=390,y=50,width=300,height=240)
GImagenROI.bind('<Button-1>', mostrar_coordenadas)
UImagen = tk.Label(ventana, background="gray")
UImagen.place(x=730,y=50,width=301,height=240)
LImagenManchas = tk.Label(ventana, background="gray")
LImagenManchas.place(x=730,y=380,width=301,height=240)
LImagenRecorte = tk.Label(ventana, background="gray")
LImagenRecorte.place(x=50,y=380,width=301,height=240)

#Cuadro de Texto
CajaTexto = tk.Text(ventana, state="disabled")
CajaTexto.place(x=1055,y=50,width=225,height=220)
CajaTexto2 = tk.Text(ventana, state="disabled")
CajaTexto2.place(x=1055,y=380,width=225,height=220)
```

Se añaden las barras para que el usuario seleccione los valores RGB para umbralizar la imagen a color.

```
#RGB se inicia en 1, ya que si no sale error de división por 0
SRedI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SRedI.place(x=400,y=620)
SGreenI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SGreenI.place(x=400, y= 660)
SBlueI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SBlueI.place(x=400, y= 700)

SRedD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SRedD.set(255)
SRedD.place(x=580,y=620)
SGreenD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SGreenD.set(255)
SGreenD.place(x=580, y= 660)
SBlueD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SBlueD.set(255)
SBlueD.place(x=580, y= 700)
```

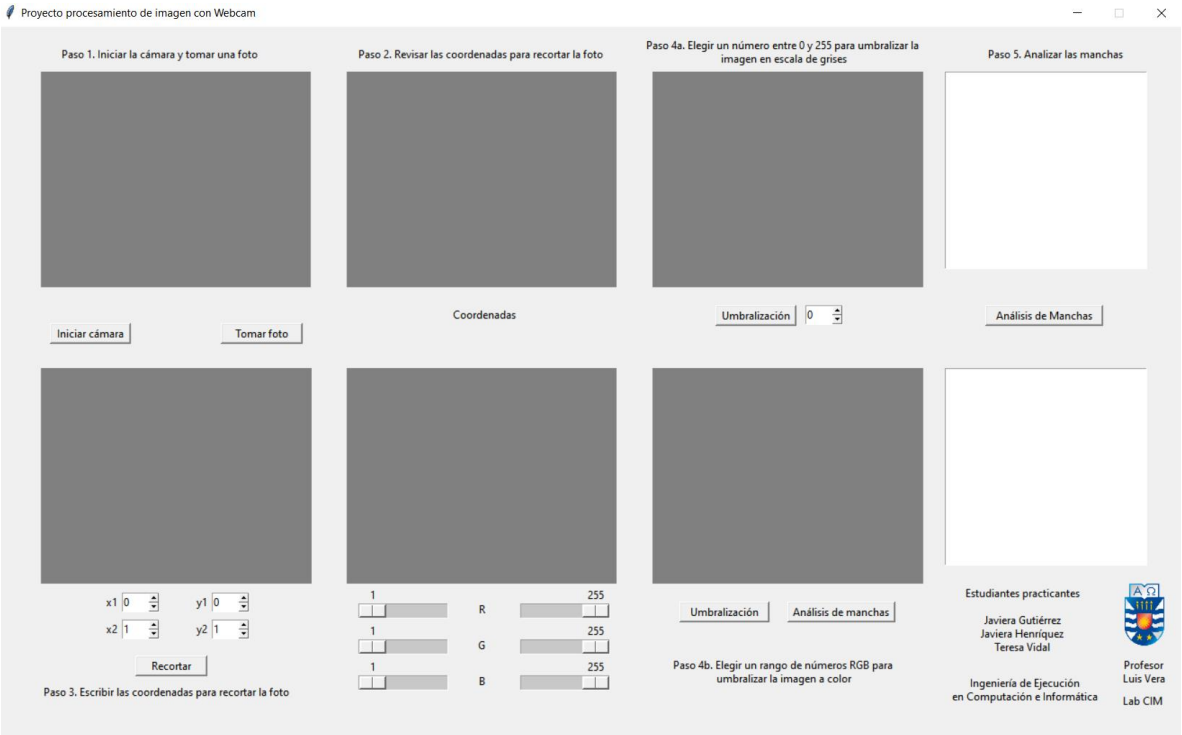
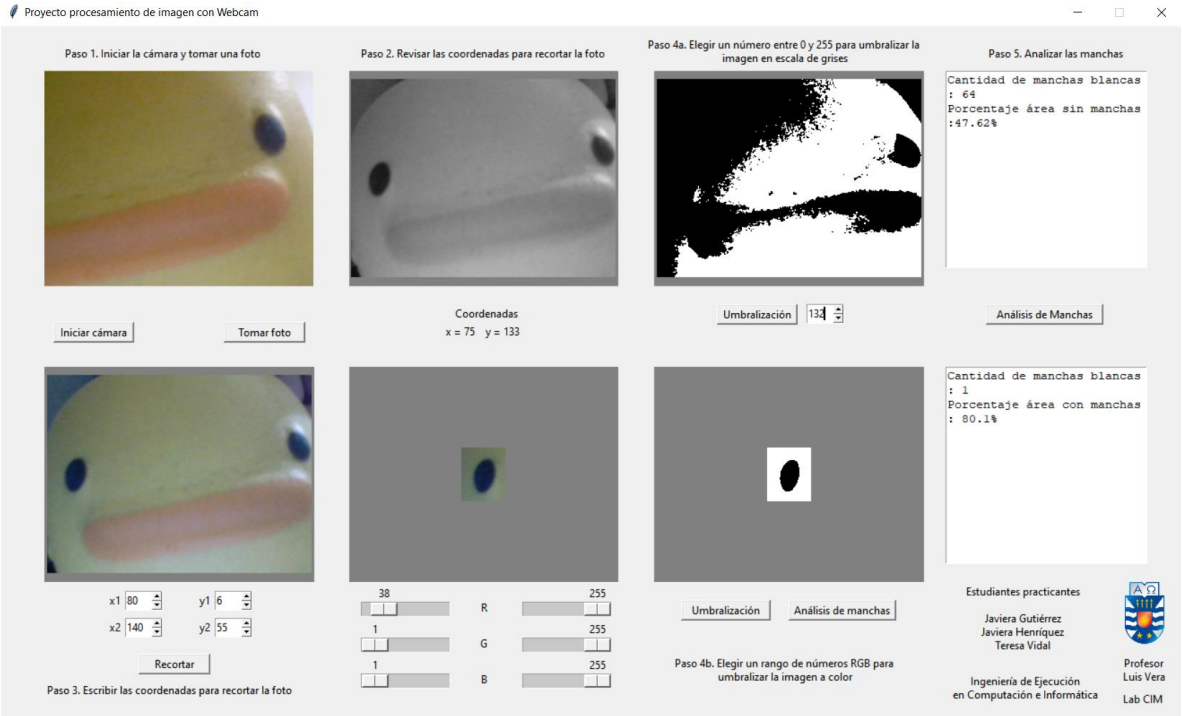
Se agregan textos para indicar los pasos a seguir para utilizar el programa.

Finalmente, se agrega el código **ventana.mainloop()** para que nuestro programa se muestre correctamente.

```
#Pasos
paso1 = tk.Label(ventana, text="Paso 1. Iniciar la cámara y tomar una foto")
paso1.place(x=70,y=20)
paso2 = tk.Label(ventana, text="Paso 2. Revisar las coordenadas para recortar la foto")
paso2.place(x=400,y=20)
paso3 = tk.Label(ventana, text="Paso 3. Escribir las coordenadas para recortar la foto")
paso3.place(x=50,y=730)
paso4a = tk.Label(ventana, text="Paso 4a. Elegir un número entre 0 y 255 para umbralizar la\n imagen en escala de grises")
paso4a.place(x=720,y=10)
paso4b = tk.Label(ventana, text="Paso 4b. Elegir un rango de números RGB para \numbralizar la imagen a color")
paso4b.place(x=750,y=700)
paso5 = tk.Label(ventana, text="Paso 5. Analizar las manchas")
paso5.place(x=1100,y=20)

ventana.mainloop()
```

Luego de compilar y ejecutar, nuestro programa se verá así:



Y al procesar una imagen, se ve así: